G.A.N.N. - Geeks Artificial Neural Network

Scientific Background & Reason for Re-Inventing the wheel in various cases

Introduction

Artificial Neural Networks (ANNs) have been dominated by backpropagation + gradient descent for decades.

While effective, this approach has limitations:

- **Differentiability requirement** – Blocks integration of non-smooth, logical or stochastic units
- **Memory-intensive backward pass** – Scaling issues on low-power or edge hardware
- **Over-parameterization reliance** – Large trial-and-error architectures instead of principled priors

G.A.N.N. proposes an alternative paradigm:

- **Tree-based neuron graph** (not a fixed layer stack)
- **Adaptive baselines** - instead of fixed thresholds!
- **Derivative-free optimization** (ES, SPSA, CMA-ES) instead of strict back-propagation
- **Topology heuristics** to constrain architecture search efficiently

Neuron Model & Baseline / Threshold Mechanism

Each neuron computes an affine transform with an activation as described in the original documentation.

$$s_j = \sum_{i \in \mathcal{P}(j)} w_{ij} x_i + b_j, \quad y_j = \phi_j(s_j)$$

- $\mathcal{P}(j)$ = parent nodes of neuron j.
- ϕ_j = activation (ReLU, GELU, sign, step, etc.).
- Network graph = **directed acyclic graph (DAG)**, not strictly layered.

Instead of a fixed threshold, each neuron computes a **dynamic baseline**.

$$\text{base}_j = \frac{1}{|\mathcal{P}(j)|} \sum_{i \in \mathcal{P}(j)} w_{ij} x_i \;=\; \frac{s_j}{d_j}$$

So, neurons fire, based on deviation from their local mean input (homeostatic centering)

The purpose is to normalize for fan-in size, center activations and stabilize step-based neurons. Alternatively we could use **EMA** (Exponential Moving Average) baseline. In this case it would look something like this:

$$\text{base}_t = (1 - \beta)\,\text{base}_{t-1} + \beta\, s_t$$

s_t = the neuron's current summed input at time t.
$\beta \in (0, 1)$ = smoothing factor (e.g., 0.01 → slow update, 0.1 → faster adaptation).
base_{t-1} = previous baseline value.

In this case each neuro has more autonomy but it stresses the randomness (entropy) even higher / which is counter intuitive for most cases.

To summarize, the base rule is not meant as a biological claim, but as a computational heuristic for scale-invariance, centering and stability. In practice, EMA or learned baselines are preferred, but the original version kept G.A.N.N. lightweight and embeddable when it was first implemented in 2006. I am open to reconsider and further improve various aspects in 2025.

Learning Without Backpropagation

Instead of chain-rule gradients, G.A.N.N. applies **zeroth-order stochastic estimators**.

A. Evolution Strategies (ES)

$$\hat{g} = \frac{1}{m\sigma} \sum_{k=1}^{m} \left[L(\theta + \sigma \epsilon_k) - L(\theta - \sigma \epsilon_k) \right] \epsilon_k$$

$$\theta \leftarrow \theta - \alpha \hat{g}$$

B. SPSA (Simultaneous Perturbation)

$$\hat{g}_i = \frac{L(\theta + c\Delta) - L(\theta - c\Delta)}{2c\,\Delta_i}, \quad \Delta_i \in \{-1, +1\}$$

$$\theta \leftarrow \theta - a\,\hat{g}$$

Topology Heuristics

G.A.N.N. **DOES NOT** claim "*optimal topology before training*"**.** Instead, it defines priors to constrain search space!

The *"a priori*" step in G.A.N.N. is not a claim of omni-science. It is a **bounded search initialization strategy** guided by input/output scale, task entropy, data redundancy and hardware limits. Actual optimization still occurs through training and pruning.

The technique and mathematical approach can be mapped as:

Let $z = (n_{\text{in}}, n_{\text{out}}, \hat{C}, \hat{H}, B)$:

- $n_{\text{in}}, n_{\text{out}}$: input/output dimensions.
- $\hat{C}$: task complexity proxy (label entropy, #classes).
- $\hat{H}$: intrinsic input dimension (PCA rank, variance explained).
- B: resource budget (compute, memory).

Functions:

- Depth prior:

$$O(z) = 1 + \mathbf{1}[\hat{H} > \tau_H] + \mathbf{1}[\hat{C} > \tau_C]$$

- Width heuristic:

$$w_\ell = \text{clip}(k\sqrt{n_{\text{in}} n_{\text{out}}}, w_{\min}, w_{\max})$$

The process is like:

1. Build seed topology using O, D, A
2. Mutate & prune graph with ES/CMA-ES
3. Optionally fine-tune weights with backprop (hybrid mode)

***** ***G.A.N.N does not utilize ReLU/GELU or other similar functions in 2006.***

Training Workflow

1. Initialize seed topology with heuristics
2. Train using ES/SPSA for global optimization
3. Hybrid refine with SGD/Adam (optional)
4. Prune redundant connections for efficiency

This balances:

- Global exploration / derivative-free
- Local exploitation / gradient-based

Advantages of G.A.N.N.

- Works with **non-differentiable modules** (logic ops, quantized hardware)
- **No backward pass** - memory savings, edge deployment
- **Topology adaptation** - avoids blind over-parameterization
- Can be combined with **back-propagation** for hybrid optimization

Conclusion

G.A.N.N. is not a rejection of backpropagation. It is a **generalization** of neural learning to settings where backprop is:

- Impractical (edge devices, hardware constraints)
- Inapplicable (non-differentiable ops)
- Inefficient (large-scale exploratory search)

It provides mathematically grounded, implementable methods while keeping open the option for hybridization with standard techniques which also is very optimal for **NON-GPU** implementations. In fact, G.A.N.N. runs smoothly in various embedded / micro-controllers.

Best,
George Delaportas (g0d/ViR4X)
Creator of G.A.N.N.
https://github.com/g0d
g.delaportas@gmail.com